Séance 07

Un système performant

Qu’est ce qui caractéristique un système performant ? Et comment le mettre en oeuvre

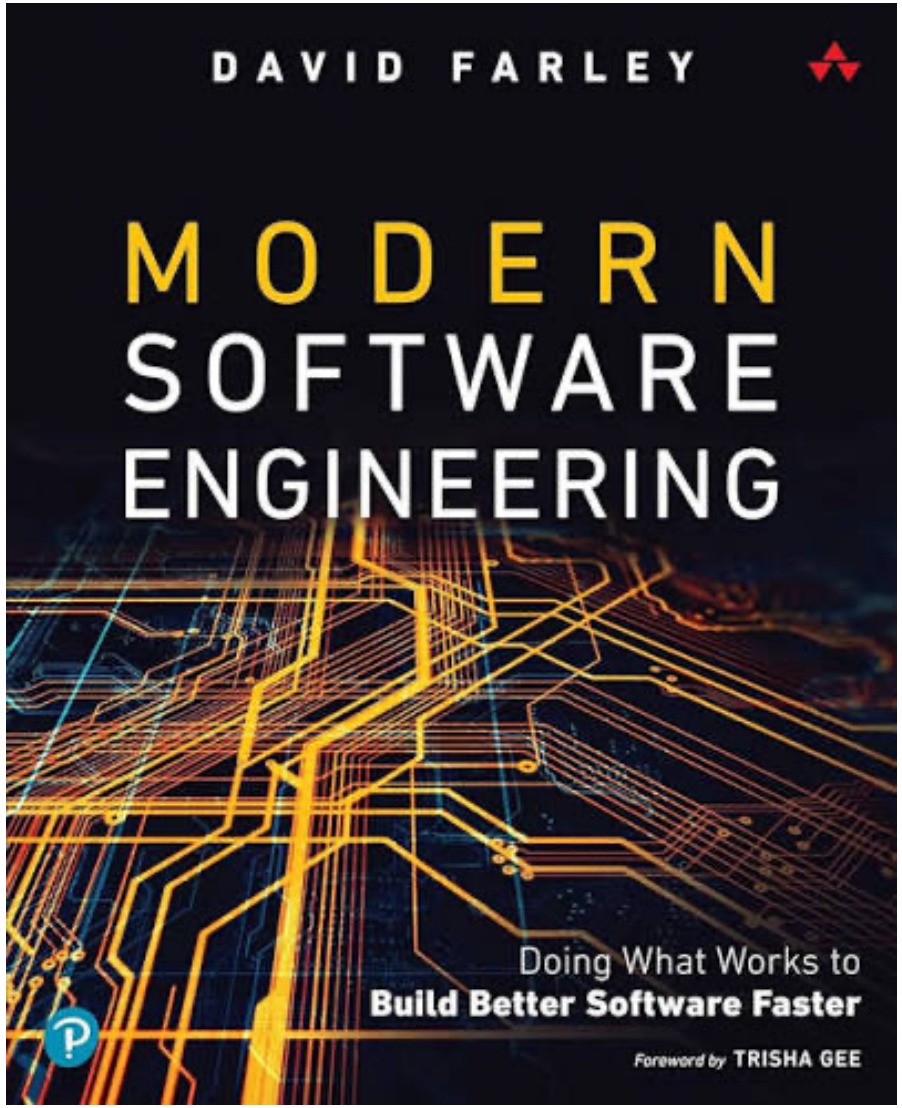
01 – Définir la performance

La performance en développement logiciel

Une métrique impossible à définir ?

- *Most metrics applied to software development are either irrelevant (velocity) or sometimes positively harmful (lines of code or test coverage).*

- Productivity, of course, is something you determine by looking at the input of an activity and its output. So to measure software productivity you have to measure the output of software development - the reason we can't measure productivity is because we can't measure **output**.



https://martinfowler.com/bliki/CannotMeasureProductivity.html

The SPACE of Developer Productivity : 5 dimensions de la productivité

https://queue.acm.org/doi/10.1145/3454122.3454124

Dimension	Ce qu'elle mesure
S — Satisfaction & Well-being	Satisfaction, motivation, bien-être et risque de burnout
P — Performance	Qualité, impact et valeur du travail réalisé
A — Activity	Activités observables : commits, PR, code reviews, etc.
C — Communication & Collaboration	Collaboration, partage de connaissances et coordination
E — Efficiency & Flow	Capacité à travailler efficacement, sans interruptions ni frictions

FIGURE 1: **EXAMPLE METRICS**

LEVEL	SATISFACTION & WELL-BEING How fulfilled, happy, and healthy one is	PERFORMANCE An outcome of a process	ACTIVITY The count of actions or outputs	COMMUNICATION & COLLABORATION How people talk and work together	EFFICIENCY & FLOW Doing work with minimal delays or interruptions
INDIVIDUAL One person	*Developer satisfaction *Retention† *Satisfaction with code reviews assigned *Perception of code reviews	*Code review velocity	*Number of code reviews completed *Coding time *# Commits *Lines of code†	*Code review score (quality or thoughtfulness) *PR merge times *Quality of meetings† *Knowledge sharing, discoverability (quality of documentation)	*Code review timing *Productivity perception *Lack of interruptions
TEAM OR GROUP People that work together	*Developer satisfaction *Retention†	*Code review velocity *Story points shipped†	*# Story points completed†	*PR merge times *Quality of meetings† *Knowledge sharing or discoverability (quality of documentation)	*Code review timing *Handoffs
SYSTEM End-to-end work through a system (like a development pipeline)	*Satisfaction with engineering system (e.g., CI/CD pipeline)	*Code review velocity *Code review (acceptance rate) *Customer satisfaction *Reliability (uptime)	*Frequency of deployments	*Knowledge sharing, discoverability (quality of documentation)	*Code review timing *Velocity/flow through the system

† *Use these metrics with (even more) caution — they can proxy more things.*

Métrique 1 : Définir le output

Martin Fowler nous propose d'utiliser une **métrique commerciale**

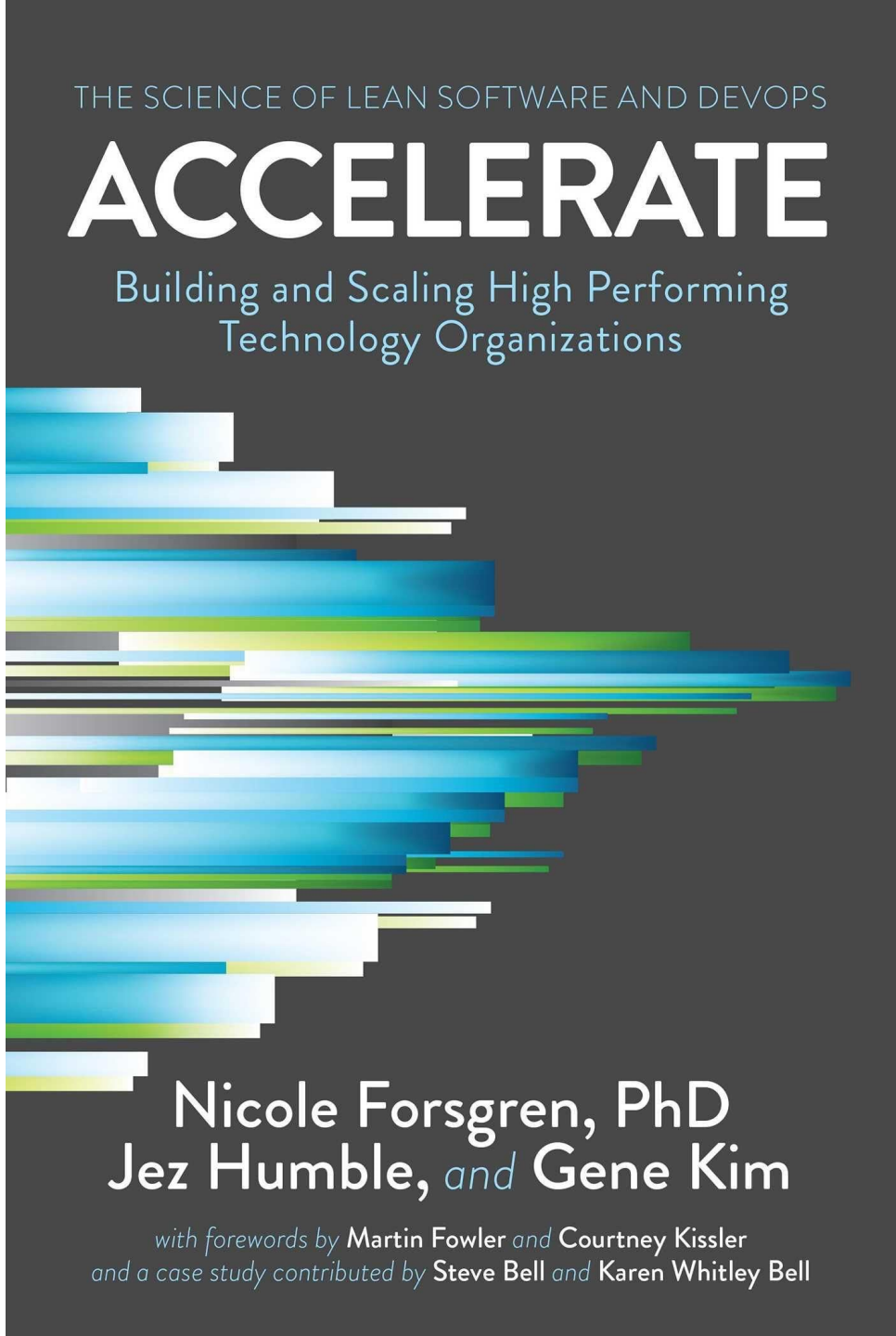
> *I assert that any true measure of software development productivity must be based on delivered business value.*
> https://martinfowler.com/bliki/CannotMeasureProductivity.html

- un développeur produit un système de 100 points de fonction en un an, dont seulement 30 sont utiles au client,
- tandis que l'autre développe un système de 50 points de fonction entièrement utiles.

DORA Métriques : Stabilité et Débit

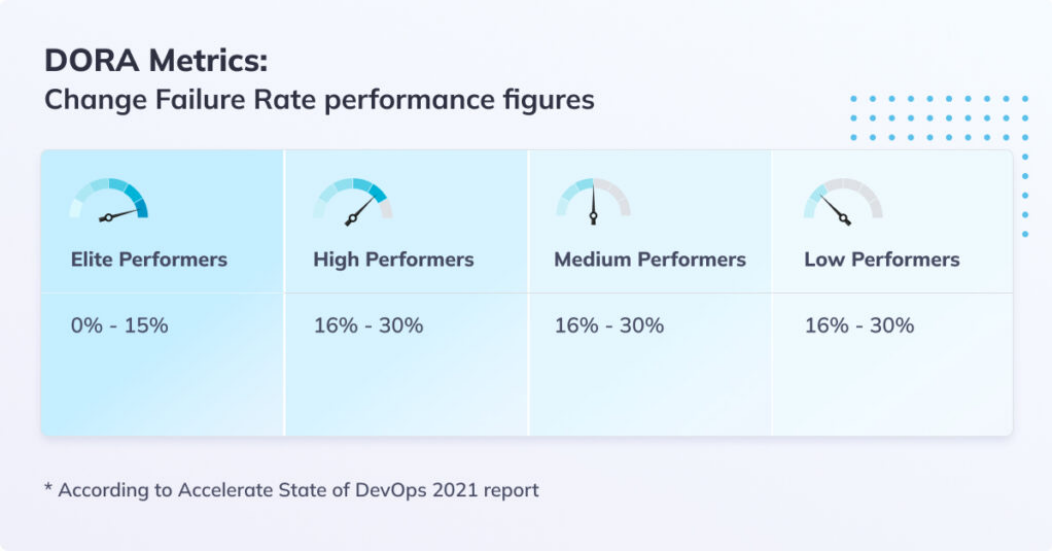
Interestingly, they don't attempt to measure productivity; rather, they evaluate the effectiveness of software development teams based on two key attributes. […] ***The measures are stability and throughput****.*

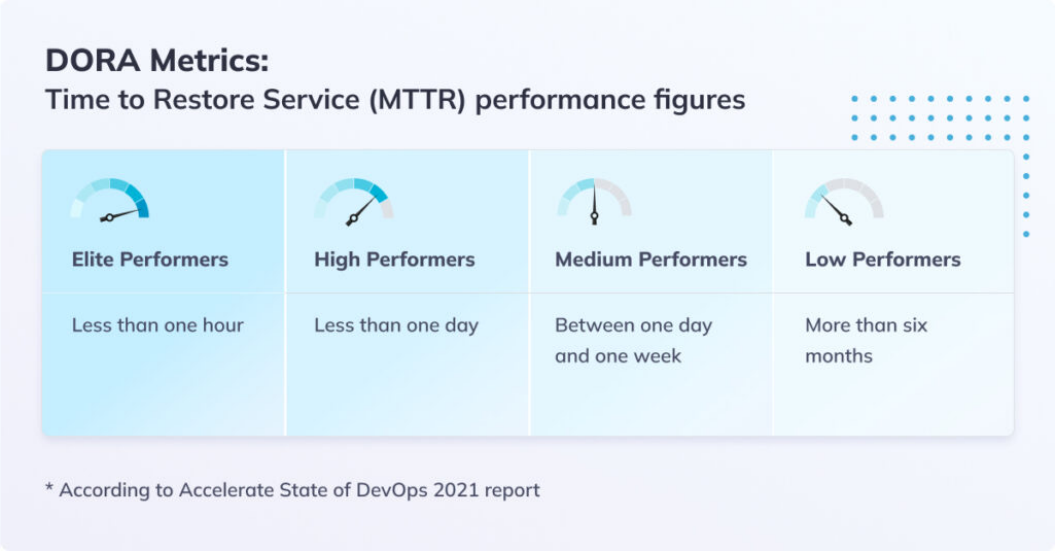
- *Teams with high stability and high throughput are classified as "high performers,"*
- *while teams with low scores against these measures are "low performers."*



Métrique 2 : la stabilité du système

- La stabilité est suivie par les éléments suivants :
  - **Taux d'échec des changements** : Taux auquel un changement introduit un défaut (bug)
  - **Temps de correction** : temps nécessaire pour corriger une défaillance.

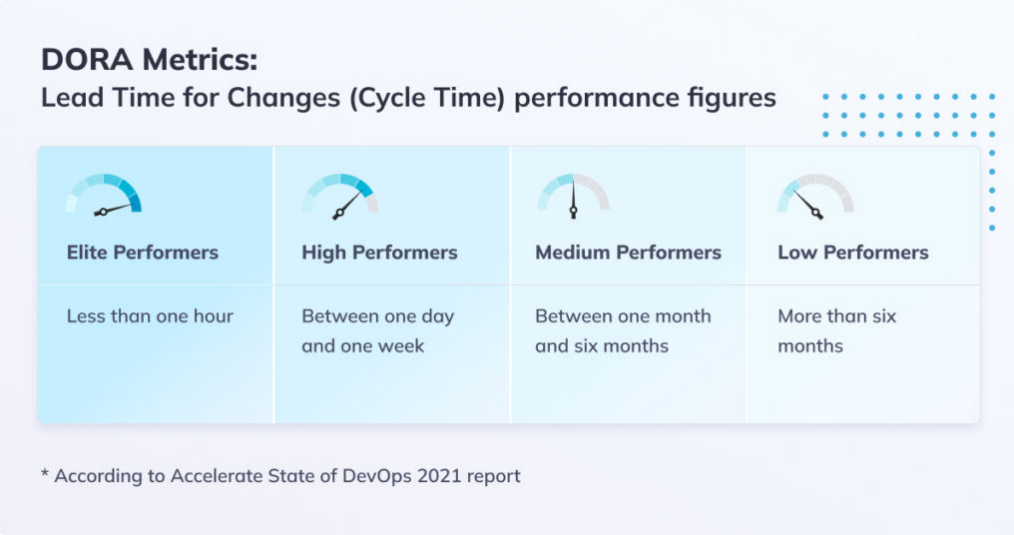


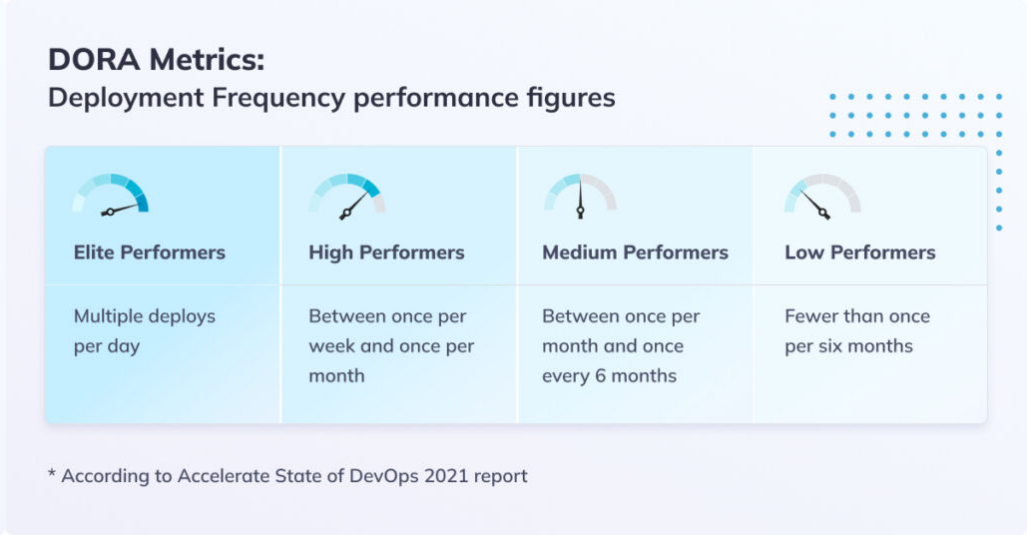


Métrique 3 : le débit du système

Le débit est suivi par les éléments suivants :

- **Le délai d'exécution (Lead Time)** : Combien de temps faut-il pour qu'une modification d'une seule ligne passe de l'état "TODO" à celui de "Done" ?
- **Fréquence** : À quelle fréquence les changements sont-ils déployés en production ?





Une approche de gestion

- Une approche de gestion qui peut être appliqué à un **processus déjà existant**

- **=> Utiliser Kanban en complément**

Méthode, méthodologie ou cadre ?

On confond souvent Kanban avec une méthodologie ou un cadre.

Dans le milieu du génie logiciel anglophone, la méthodologie correspond à une façon de définir un processus en développement logiciel et en gestion de projet (le terme peut sembler erroné, car « méthodologie » signifie plutôt « l'étude des méthodes »). Selon cette définition, les méthodologies contiennent des flux de travail et des processus normatifs définis. Elles incluent des rôles et des responsabilités bien définis et généralement spécifiques à un domaine, comme le développement de logiciels.

Un cadre de processus est, quant à lui, une méthodologie incomplète — un ensemble d'échafaudages destiné à être appliqué plus largement, mais qui nécessite une adaptation pour chaque contexte afin d'en combler les lacunes.

Kanban n'est ni une méthodologie ni un cadre de processus. Il s'agit plutôt d'une méthode, ou d'une approche de gestion, qui devrait être appliquée à un processus ou à un mode de travail déjà existant.

Il n'est pas non plus question d'opposer Kanban à une méthodologie ou un cadre donné. Il s'agit plutôt d'utiliser Kanban en complément du cadre ou de la méthode de travail en place. Kanban est destiné à vous aider à mieux gérer le travail et à améliorer la prestation de services avec l'objectif de répondre systématiquement aux attentes des clients. Kanban est un moyen d'améliorer ce que vous faites déjà et votre façon de le faire sans remplacer votre mode de travail actuel.

Conclusion : quelles métriques

Des indicateurs axés sur la valeur commerciale délivrée et sur des mesures de stabilité et de débit. Ces dernières permettent d'évaluer non seulement l'efficacité des équipes à produire du logiciel, mais aussi la qualité et l'impact des livraisons.

- Est-ce que nous construisons les bonnes choses, c'est-à-dire celles qui apportent de la valeur ? (output)
- Est-ce que ceci augmente la qualité du logiciel ? (stabilité)
- Est-ce que ceci augmente l'efficience de la création du logiciel ? (débit)

Attention

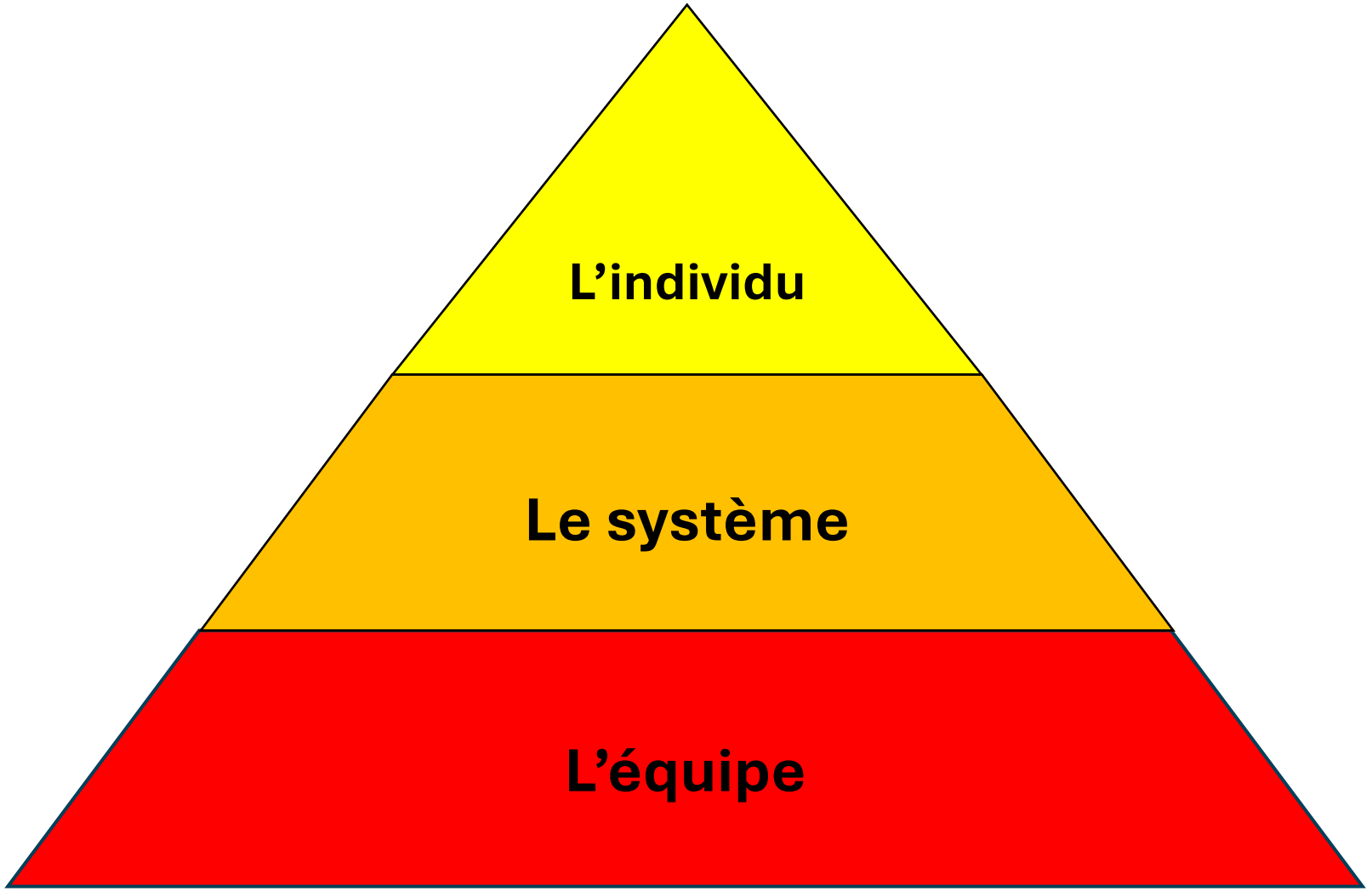
On ne mesure pas la productivité individuelle

02 – Comment faire

Comment créer de la valeur, augmenter sa stabilité et son débit ?

02.01 Les 3 niveaux où jouer

Impact faible



Impact fort

L’équipe, l’unité de performance

Les différentes pratiques et outils pour améliorer l’équipe

Les caractéristiques d’une équipe performante

- Pluridisciplinaires
- Autonome
- Transcendante

Equipe propriétaire du processus

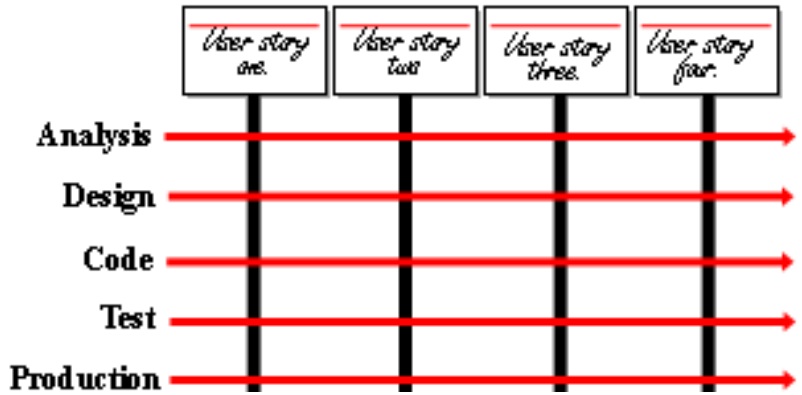
C’est l’équipe qui choisit Agile, et non la direction qui l’impose

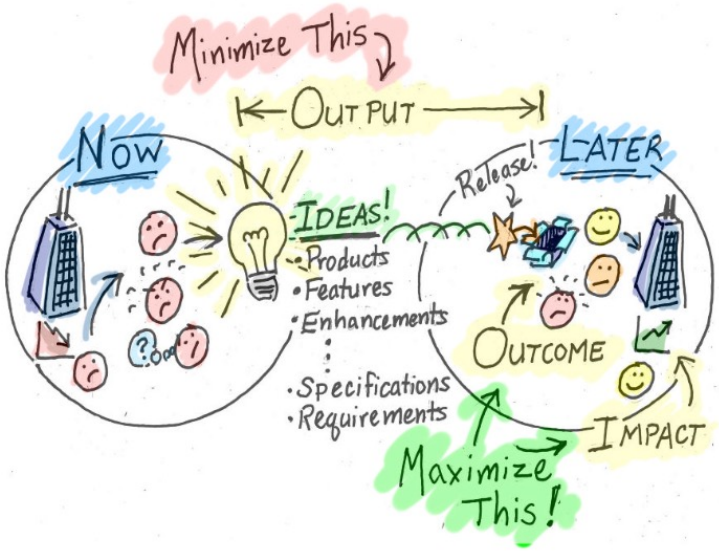
- Auto-organisation : signifie que la direction s’engage à accompagner les comportements émergeants au lieu de spécifier un comportement efficace

- Besoin d’autonomie : levier essentiel à la motivation; au lieu d’adhérer à une politique, des protocoles, les équipes disposent d’un certain degré d’autonomie
  - Comment recruter un nouveau membre
  - Comment bien former les personnes
  - Quels sont les outils nécessaires
  - …

Orienté sur la valeur métier

- A chaque itération travailler sur la plus haute priorité
  - Comment : feedback rapide du client, orienté outcome, travailler en itération courte

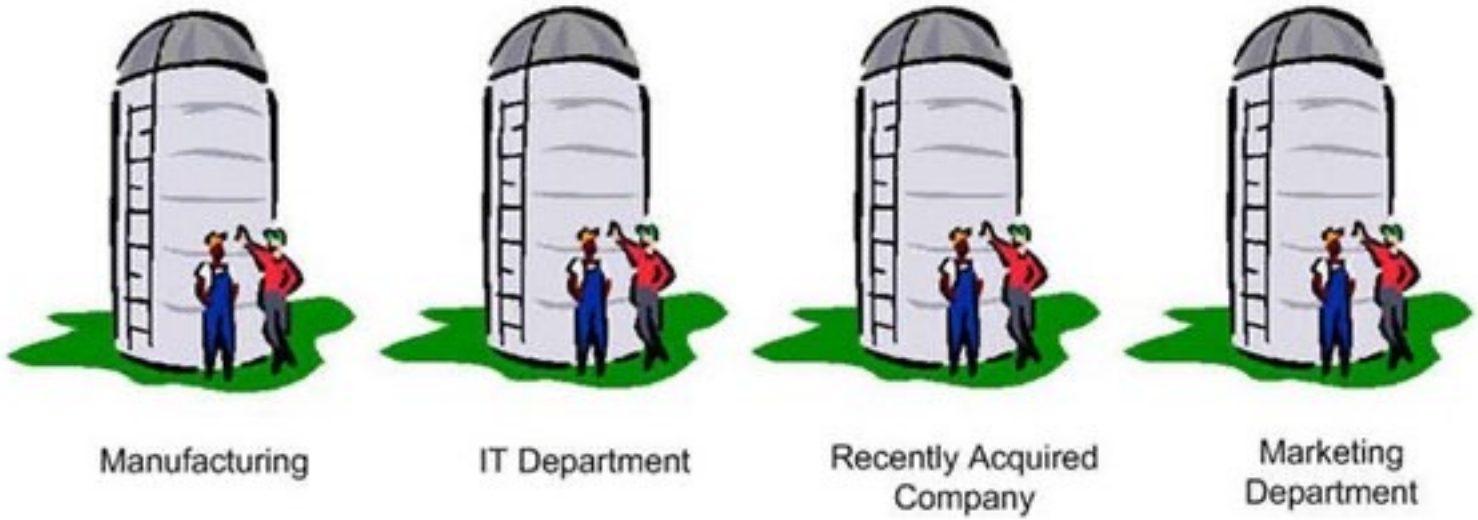




Le système doit soutenir l’équipe

Les différentes pratiques et outils pour améliorer le système

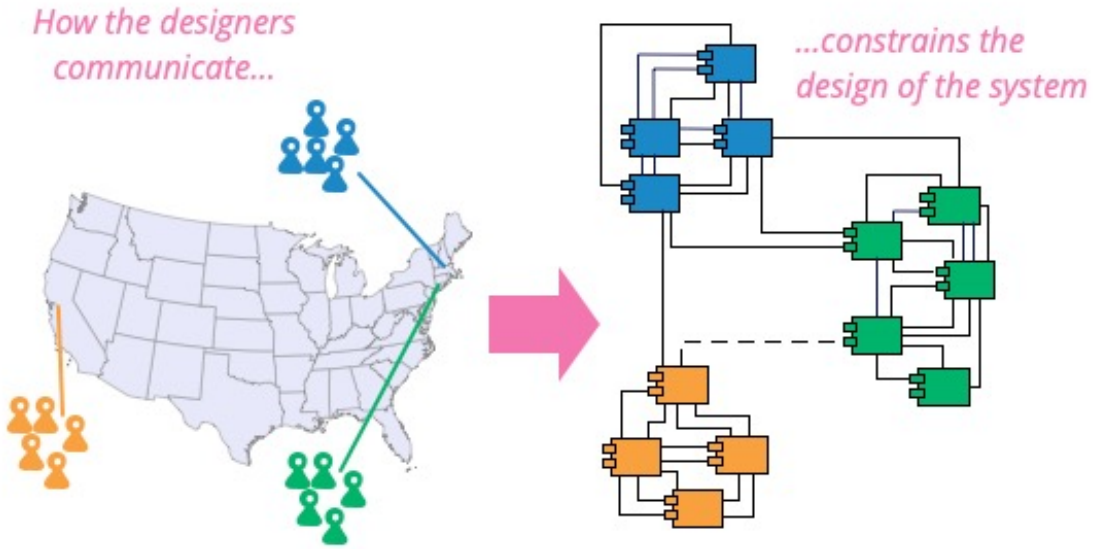
Casser les silos organisationnels

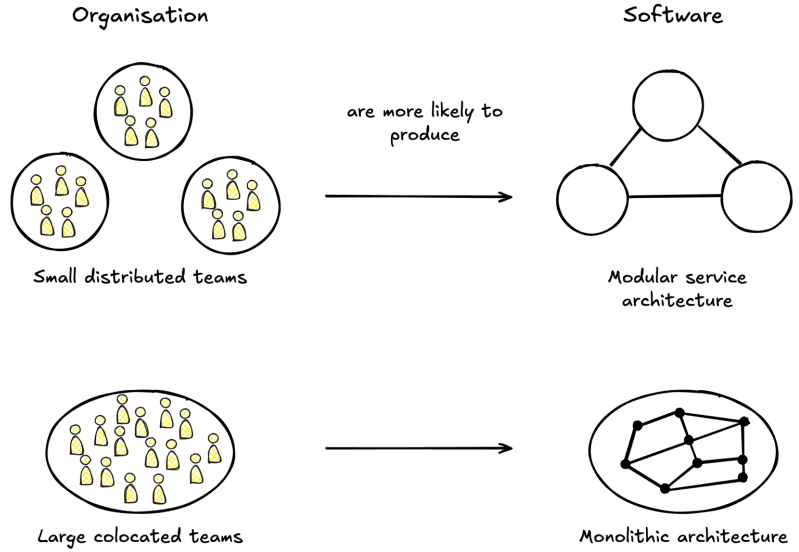


Loi de Conway

Définition

Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure.





Autrement dit, la structure des équipes est bien plus dictée par la forme du système de communication en place dans l'entreprise que par ce qui serait nécessaire ou optimal. En pratique, cela signifie que la répartition des équipes est souvent héritée d'un historique organisationnel.

Loi de Conway => Inverse Conway's Law

Définition

Change the communication patterns of the designers to encourage the desired software architecture.

- L'architecture de notre logiciel émergera de notre organisation (Conway's Law), pour transformer à moindre coût votre entreprise, modifiez ses structures de communication pour influencer l'émergence des designs d'architecture optimaux.

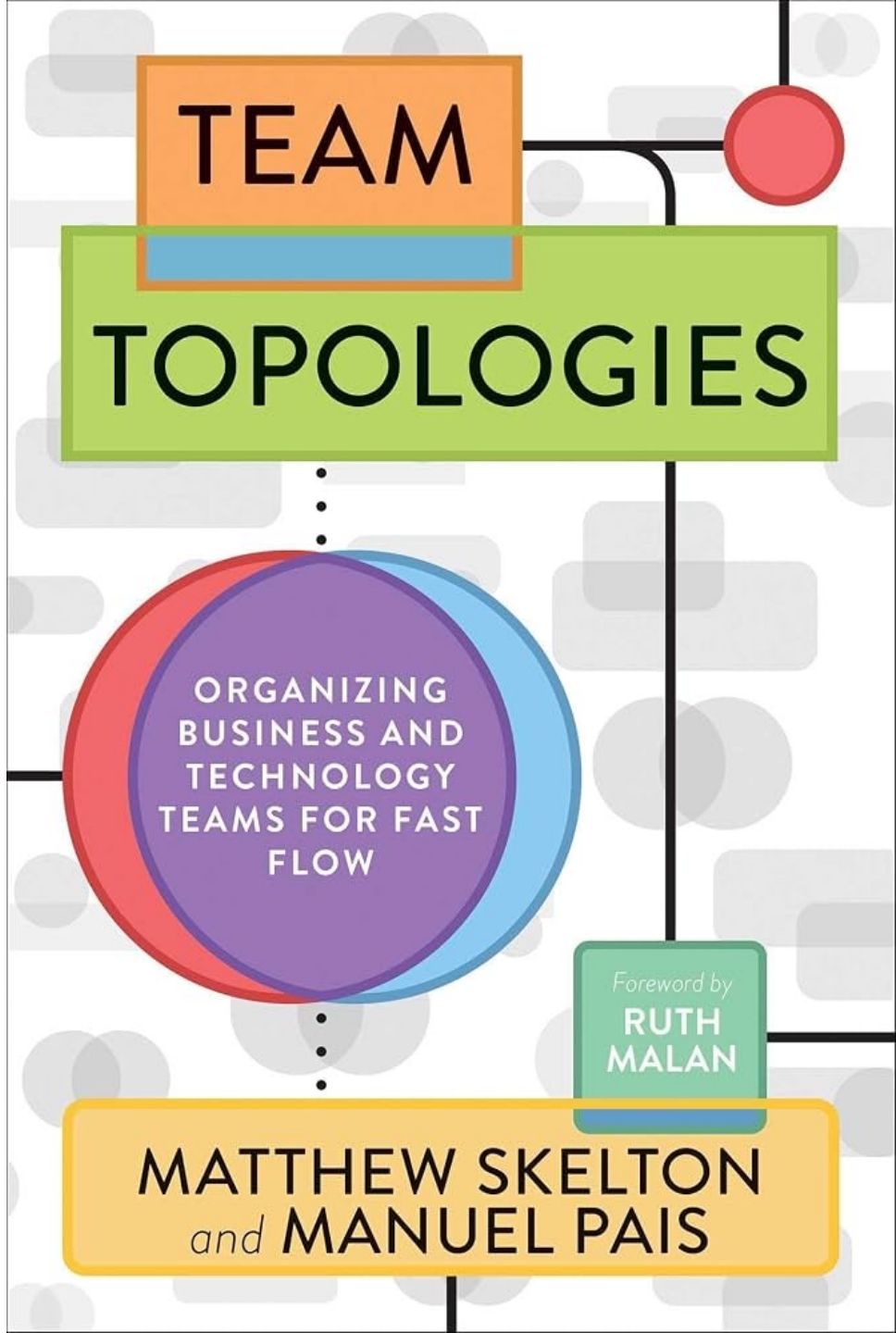
- => **En structurant les équipes pour qu'elles ressemblent à votre architecture cible, elle sera plus simple à atteindre**

Team Topologies

Comment organiser les équipes de développement pour maximiser le flux de valeur livré aux utilisateurs, sans augmenter inutilement la complexité et les coûts de coordination ?

La productivité ne dépend pas uniquement des compétences individuelles ou des pratiques de développement. Elle dépend aussi de facteurs organisationnels :

- Les dépendances entre équipes.
- La charge cognitive des développeurs.
- La clarté des responsabilités.
- La capacité à prendre des décisions de manière autonome.
- La fluidité de la livraison du logiciel.

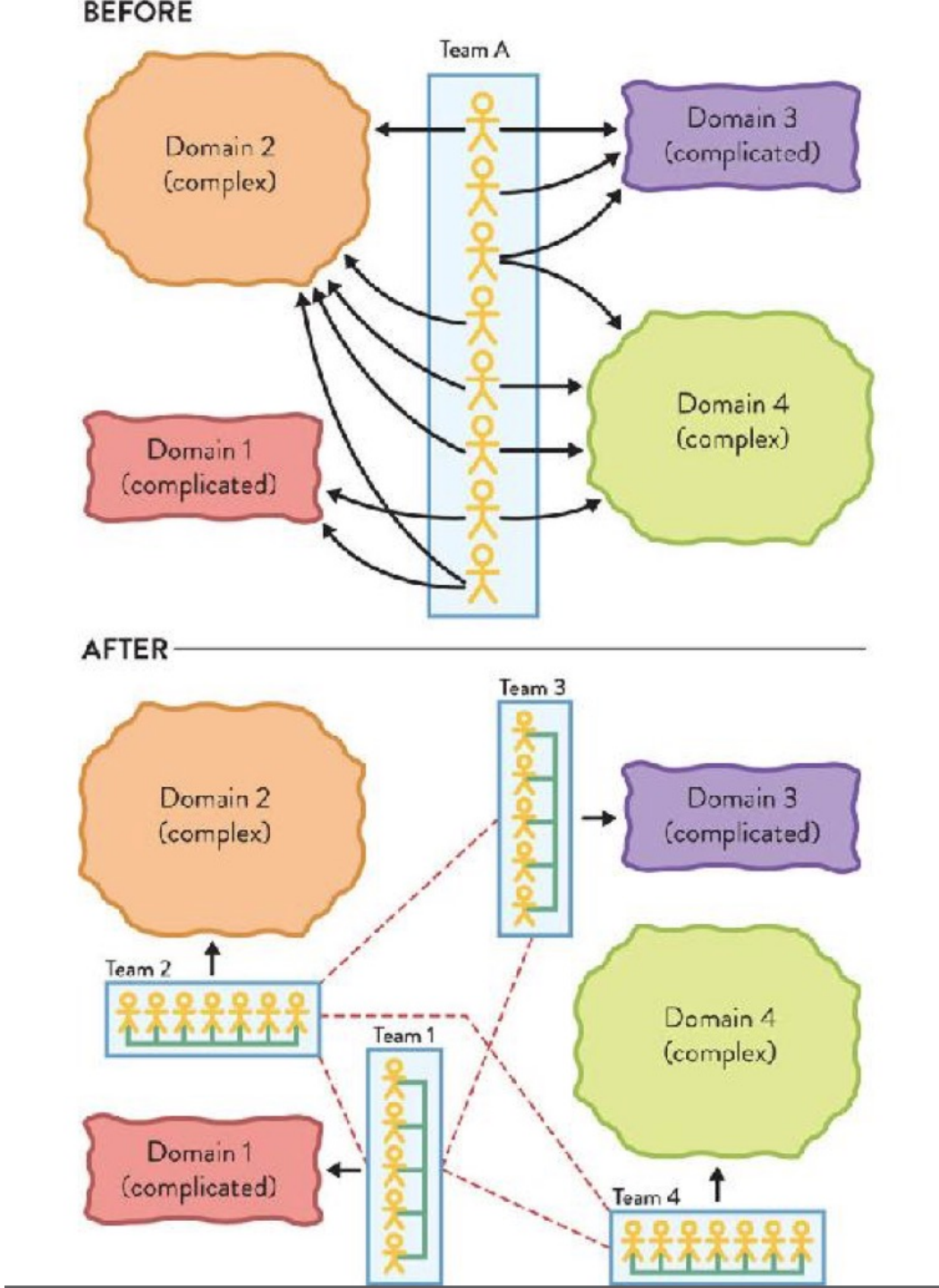


Team Topologie : réduire la charge cognitive

https://techblog.deepki.com/team_topologies/

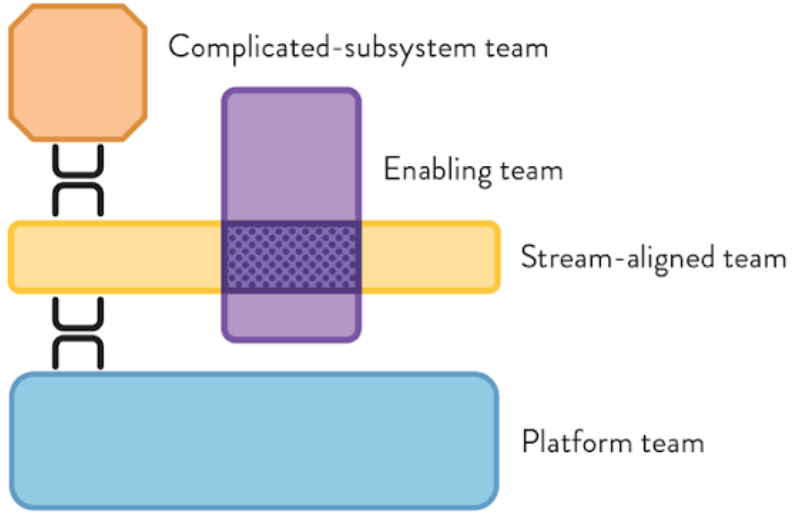
- Pourquoi : obtenir des résultats hautement qualitatifs sans les surcharger d'une quantité déraisonnable de tâches et de responsabilités.

- Comment : Limiter le nombre de domaines de responsabilité de l'équipe pour augmenter sa capacité cognitive

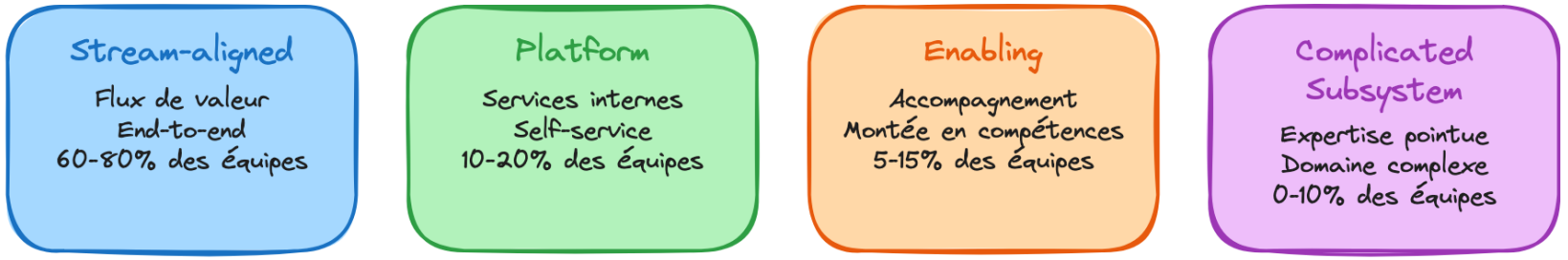


Team Topologies : 4 types d'équipe

Pour éviter une charge cognitive trop élevé



- **Stream-aligned team :** équipe autonome qui livre de la valeur de bout en bout pour un besoin métier ou utilisateur.
- **Platform team** : fournit des outils et services internes en self-service pour faciliter et accélérer le travail des autres équipes.
- **Enabling team :** accompagne temporairement les autres équipes pour les aider à monter en compétences et à surmonter des difficultés
- **Complicated Subsystem team** : prend en charge une partie techniquement complexe du système nécessitant une expertise spécialisée.



Team Topologies : 4 types d’équipe et silos

Ne sommes-nous pas entrain de re-créer des silos ?



- L'équipe Stream-aligned doit pouvoir livrer sans dépendre constamment d'autres équipes.
  - L'équipe Platform fournit une plateforme de déploiement en self-service.
  - Les développeurs peuvent déployer eux-mêmes

=> Une équipe Platform ne doit pas devenir une équipe à laquelle on délègue systématiquement le travail. Elle doit permettre aux autres équipes de réaliser ce travail de manière autonome.

Rappel

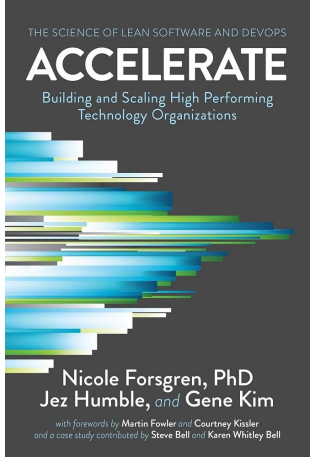
L'équipe dispose des capacités nécessaires pour livrer de la valeur sans dépendances bloquantes

Westrums Orginizational Culture

Table 3.1 Westrums Typology of Organizational Culture.

Pathological (Power-Oriented)	Bureaucratic (Rule-Oriented)	Generative (Performance-Oriented)
Low cooperation	Modest cooperation	High cooperation
Messengers "shot"	Messengers neglected	Messengers trained
Responsibilities shirked	Narrow responsibilities	Risks are shared
Bridging discouraged	Bridging tolerated	Bridging encouraged
Failure leads to scapegoating	Failure leads to justice	Failure leads to inquiry
Novelty crushed	Novelty leads to problems	Novelty implemented

	Politique (orientée Pouvoir)	Bureaucratique (Orientée Règlement)	Créative (Orientée Performances)
Coopération	Faible	Modeste	Forte
Remontée d'information	Les lanceurs d'alerte sont punis	Les lanceurs d'alerte sont négligés ou ignorés	Les lanceurs d'alerte sont encouragés et formés
Responsabilisation des opérationnels	Inexistante ou très réduite	Etroite	Les risques sont partagés
Collaboration interservices	Découragée	Tolérée	Encouragée
Réaction face à l'échec ou à l'erreur	Recherche de coupable ou de bouc émissaire	Recherche de justice	Recherche des causes
Réaction face à la nouveauté	La nouveauté est rejetée	La nouveauté est source de problèmes	La nouveauté est implémentée



Just as Dr. Westrum found in healthcare organizations, a high-trust, generative culture also predicted software delivery and organizational performance in technology value streams.

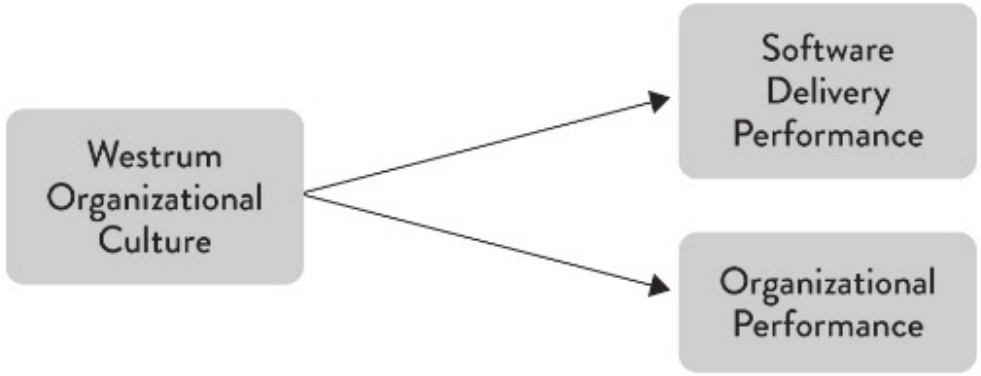


Figure 3.2: Westrum Organizational Culture's Outcomes

Westrums Orginizational Culture : viser une culture Créative

Notre objectif est de viser une culture “Generative”; les principales caractéristiques d’une équipe générative sont :

- une forte coopération
- Partage du risque -> **Psychological Safety**
- une réaction irréprochable aux échecs
- encourager la prise d’initiative
- mise en œuvre des nouveautés (technologique par exemple)

Westrums Orginizational Culture : comment l’obtenir ?

Affirmation

la façon de changer la culture d’une organisation **n’est pas de commencer par changer la façon dont les gens pensent, mais plutôt de commencer par changer la façon dont les gens se comportent** – ce qu’ils font

Le changement culturel viendra d'un changement de pratiques

Westrums Orginizational Culture : comment l'obtenir ?

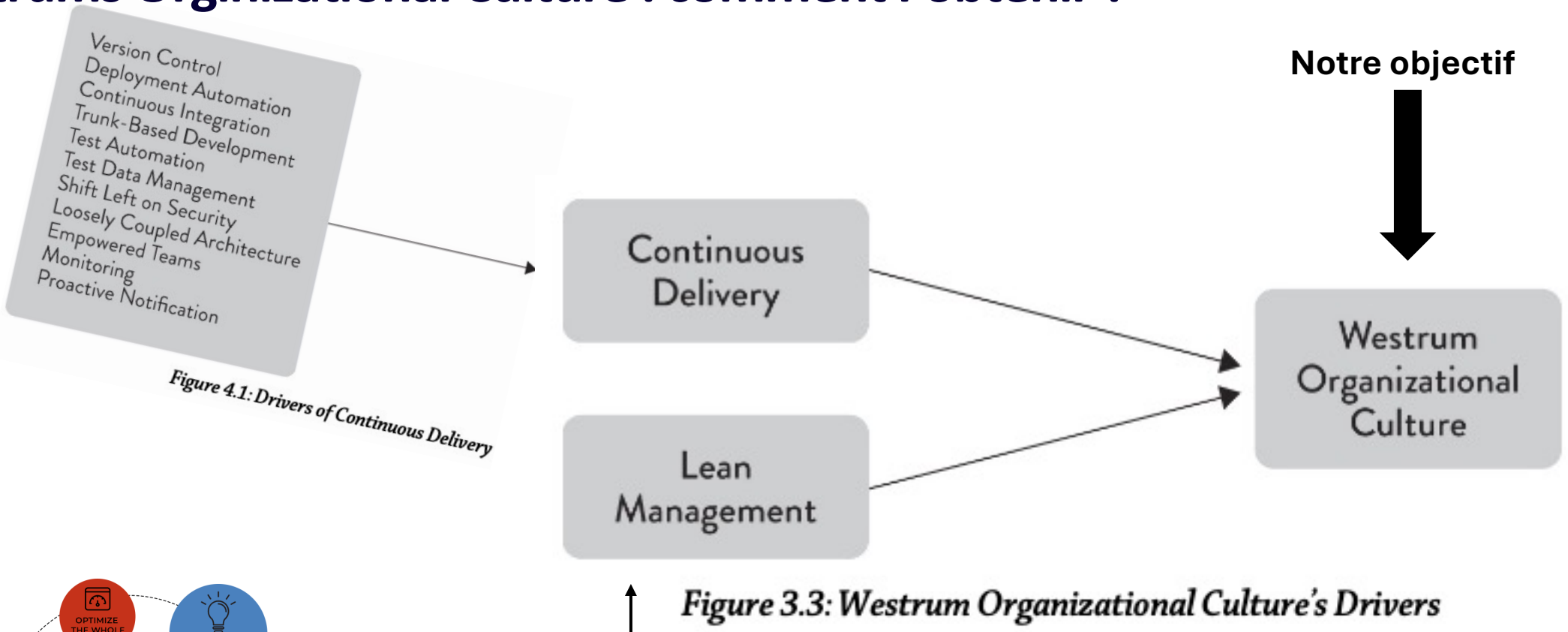


Figure 4.1: Drivers of Continuous Delivery

Figure 3.3: Westrum Organizational Culture's Drivers



Feedback, itération, élimination gaspillage,

Decision latency

Définition

combien de temps s'écoule entre le fait qu'une décision doit être prise et le moment où elle est réellement prise.

> *So he started adding the question "How fast do you make decisions?" to his research. And when he looked at the projects that had problems, he realized that it wasn't an aberration, it was commonplace.* ***In projects that failed, people simply weren't making decisions.***

- Scrum délègue la prise de décision au niveau de l'équipe. Il n'y a en réalité que deux décideurs :
  - le Product Owner
  - et l'équipe

Value Stream Map

Identifier les sources de gaspillage sur la chaine de production

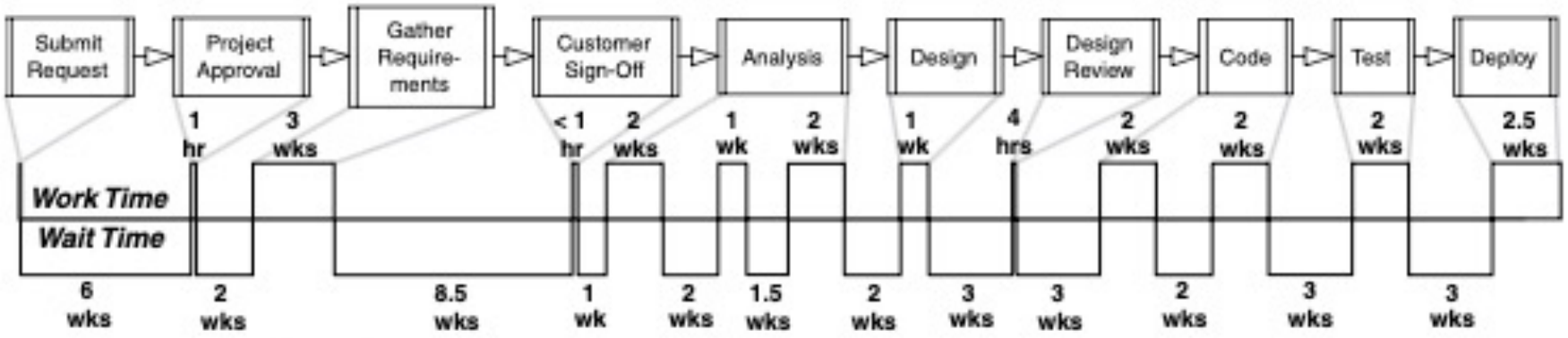


Figure 1.2 *Traditional value stream map.*

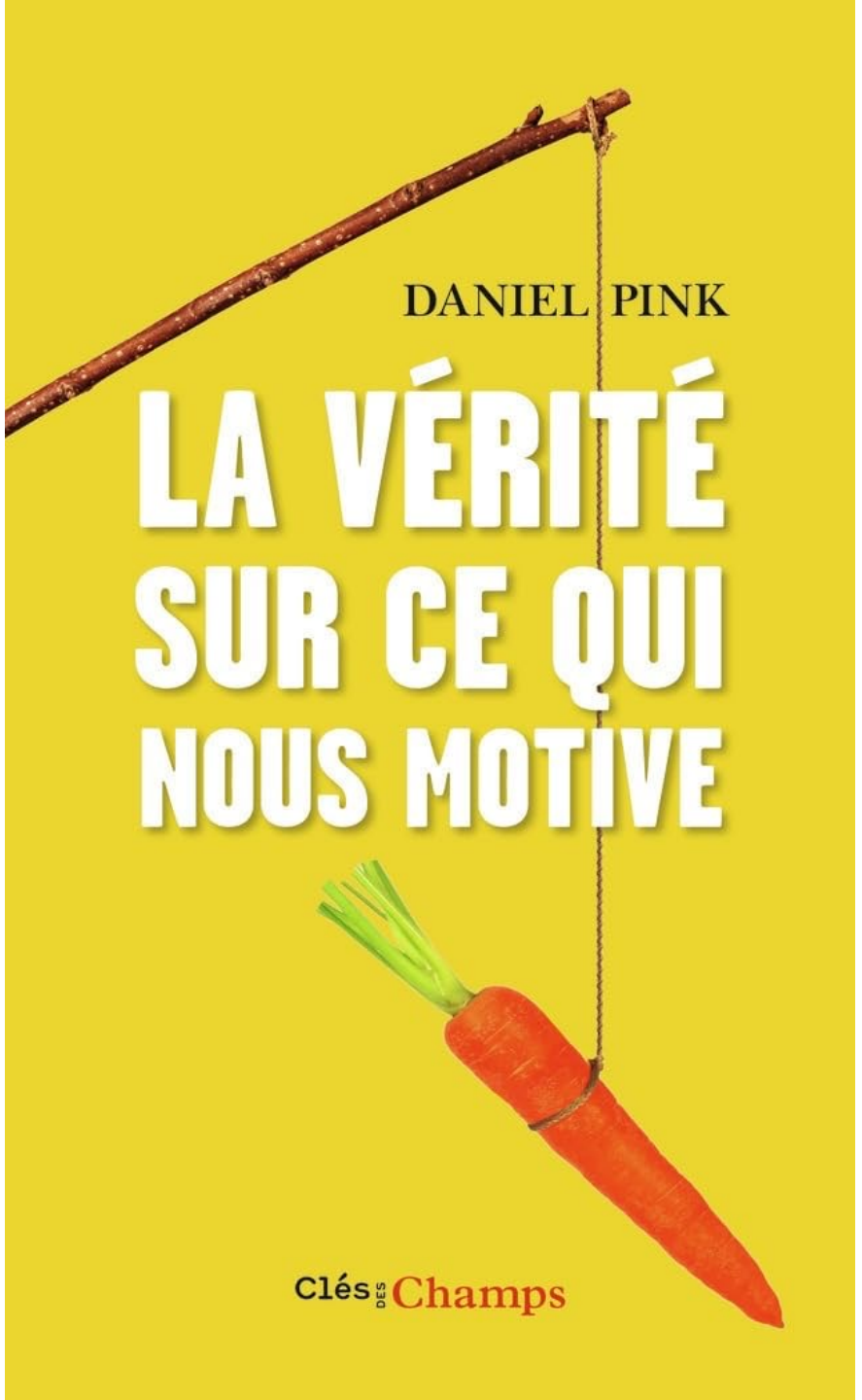
L’individu

Ne pas s’y consacrer en premier !

Qu’est ce qui motive un individu

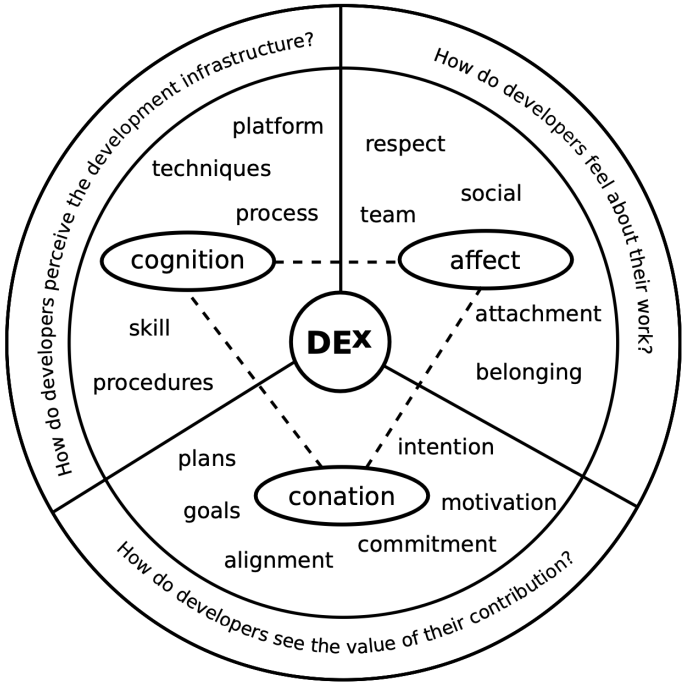
Daniel Pink rappelle les trois éléments de la motivation personnelle qui peuvent être impactés par une charge cognitive trop importante :

- l’autonomie (trop nombreuses demandes et des priorités confuses)
- la maîtrise (touche-à-tout, maître de rien)
- le but (trop de domaines de responsabilité)



La Développer Experience (DevEx)

- La Developer Experience (DevEx) désigne l'expérience vécue par les développeurs dans leur environnement de travail.



- Elle repose sur trois dimensions principales :
  - Feedback Loops : rapidité des retours (tests, CI/CD, code reviews).
  - Cognitive Load : réduction de la complexité et de la charge mentale.
  - Flow State : capacité à travailler avec concentration, sans interruptions.

La Développer Experience (DevEx)

FIGURE 1: **EXAMPLE METRICS**

LEVEL	SATISFACTION & WELL-BEING How fulfilled, happy, and healthy one is	PERFORMANCE An outcome of a process	ACTIVITY The count of actions or outputs	COMMUNICATION & COLLABORATION How people talk and work together	EFFICIENCY & FLOW Doing work with minimal delays or interruptions
INDIVIDUAL One person	★Developer satisfaction ★Retention† ★Satisfaction with code reviews assigned ★Perception of code reviews	★Code review velocity	★Number of code reviews completed ★Coding time ★# Commits ★Lines of code†	★Code review score (quality or thoughtfulness) ★PR merge times ★Quality of meetings† ★Knowledge sharing, discoverability (quality of	★Code review timing ★Productivity perception ★Lack of interruptions